



MODULE 8

Data & API Integration

Phase 02 | iOS Development with Swift

Module 8 Roadmap

PART 1

Networking Fundamentals

- REST & HTTP
- URLSession & Async/Await
- JSON & Codable
- Error Handling

PART 2

Auth, Security & Persistence

- Authentication Workflows
- Secure Token Handling
- Local Persistence Overview
- Core Data Fundamentals I

PART 3

Core Data & Caching

- Core Data Fundamentals II
- Caching Strategies
- Module Exercise



RESTful API Fundamentals & HTTP

- HTTP Verbs & Status Codes
- REST Conventions
- Reading API Documentation



HTTP Verbs & Status Codes

GET

Retrieve data, no side effects

`GET /accounts`

POST

Create something, or trigger an action

`POST /transfers`

PUT / PATCH

Update an existing resource

`PATCH /accounts/{id}`

DELETE

Remove a resource

`DELETE /payees/{id}`

Status Code Families

2xx

Success

4xx

Client error - you sent something wrong

5xx

Server error - not your fault, but you must handle it



Reading API Documentation



Base URL

The root the whole API is hosted under - changes per environment (recall Module 5's Dev/Staging/Prod schemes)



Endpoint & path parameters

The specific resource, e.g. `/accounts/{accountId}/transactions`



Headers

Metadata about the request - Authorization, Content-Type, Accept



Request / response body shape

The JSON contract both sides agree to - this is what Codable models in Lesson 3 will mirror exactly



Lesson 1 - RESTful API Fundamentals & HTTP

Skill Check ✓

- Which HTTP verb would you use to submit a new money transfer, and which to simply view account balances?
- A response comes back with status 404. Which status code family is that, and what does it generally mean?
- Name the four things you need from an API doc page before you can write a single line of networking code.



URLSession & Async/Await

- URLSession Fundamentals
- Swift Concurrency: async/await
- Your First Real Network Call



URLSession

- URLSession is Apple's native networking framework
 - It is built into the Foundation library
 - It is available for iOS, macOS, watchOS, and tvOS
- It provides an API for interacting with web servers via HTTP and HTTPS
- URLSession allows apps to download data, upload files and data, and fetch assets
- It works completely asynchronously
- URLSession (the architecture) is an entire cooperative ecosystem consisting of five parts:
 - URLSession (the coordinator) manages and kicks off network operations
 - URLSessionConfiguration (the settings) configures timeouts, cookie policies, caching, etc.
 - URLRequest (the request) represents the network request package - URL, HTTP method, headers, etc.
 - URLSessionTask (the action) is the worker created by the session to run the request
 - URLResponse (the metadata) contains the server's response details such as headers and status codes



Types of Network Tasks

- When you configure a session, you create specific tasks depending on your objective
- **Data Tasks** are best for sending or receiving short pieces of information
 - e.g. interacting with a RESTful API
 - Format is usually JSON
- **Download Tasks** are designed to pull large files (like videos, images, or PDFs) directly into the device's storage instead of caching them in memory
 - These tasks can pause and resume safely if a connection drops
- **Upload Tasks** are handcrafted to stream or push local data and files onto a server



Creating the URL

- The URL initializer function returns an `Optional<URL>`
 - It must be unwrapped for use in a request
- If your code has the url string as a literal string that you can guarantee is well-formed then you could force unwrap it
- If not, then one of the safe unwrapping patterns should be used
 - e.g. if the user is entering the URL or you are retrieving it from a data store

Code Bite

```
let url = URL(string:
    "https://api-dev.pncmobile.com/accounts")!

guard let url = URL(string: urlStr) else {
    return
}
```



Creating the URLRequest

- URLRequest is a structure that stores all the settings for an HTTP request
- It is created by passing a URL object into its initializer
- The httpMethod property should be set to a valid HTTP method
- HTTP Headers can be specified using the setValue() method

Code Bite

```
var request = URLRequest(url: url)

request.httpMethod = "GET"

request.setValue("Bearer \token)",
    forHTTPHeaderField: "Authorization")
```



Encoding Data for POST, PUT, or PATCH

- The URLRequest's httpBody property contains data that will be sent to the server
- The most common format is JSON
- Swift's JSONEncoder will convert a dictionary into the proper format

Code Bite

```
request.httpMethod = "POST"
request.setValue("application/json",
    forHTTPHeaderField: "Content-Type")

let bodyData = try JSONEncoder()
    .encode(["name": "Alice"])

request.httpBody = bodyData
```



Sending the Request

- The `URLSession.shared` static property contains the singleton `URLSession` object
- Its `data()` method is an async method used to fetch data from a given URL
- It is marked *async* and *throws* so you must call it using *try await* inside an asynchronous context
- It returns a tuple containing the raw binary data response and a `URLResponse` object

Code Bite

```
let (data, response) = try await
    URLSession.shared.data(for: request)

guard let httpResponse = response as?
    HTTPURLResponse,
    (200...299).contains(httpResponse
        .statusCode) else {
    throw NetworkError.badResponse
}
```



A Complete Request

```
let url = URL(string: "https://api-dev.pncmobile.com/accounts")!
var request = URLRequest(url: url)
request.httpMethod = "GET"
request.setValue("Bearer \$(token)", forHTTPHeaderField: "Authorization")

let (data, response) = try await URLSession.shared.data(for: request)

guard let httpResponse = response as? HTTPURLResponse,
      (200...299).contains(httpResponse.statusCode) else {
    throw NetworkError.badResponse
}

// 'data' is now the raw response bytes - ready for Codable from Lesson 3
```



Swift Concurrency: traditional style

// Old style - completion handlers

```
func fetchAccounts(completion: @escaping ([Account]?, Error?) -> Void) {  
    URLSession.shared.dataTask(with: url) { data, _, error in  
        if let error { completion(nil, error); return }  
        guard let data else { completion(nil, NetworkError.noData); return }  
        let accounts = try? JSONDecoder().decode([Account].self, from: data)  
        completion(accounts, nil)  
    }.resume()  
}
```

// Nests badly once a second dependent call is needed



Swift Concurrency: async/await

```
// Modern style - async/await  
func fetchAccounts() async throws -> [Account] {  
    let (data, _) = try await URLSession.shared.data(for: request)  
    return try JSONDecoder().decode([Account].self, from: data)  
}  
// Reads top-to-bottom like synchronous code. Errors propagate with 'throws'.
```



Lesson 2 - URLSession & Async/Await

Skill Check ✓

- What problem does async/await solve that completion-handler-based networking code has?
- What does the 'await' keyword actually do at the call site?
- Why check response as? HTTPURLResponse and its statusCode before trusting the returned data?



JSON Parsing with Codable

- Codable & Automatic Synthesis
- CodingKeys & Snake Case
- Nested & Optional Fields



Codable: Automatic JSON ↔ Swift Conversion

- Create your Swift model to conform to the Codable protocol
 - This combines two other protocols: Encodable and Decodable
- Note that snake_case in JSON becomes camelCase in Swift with zero extra code
 - When using .convertFromSnakeCase

Code Bite

```
// JSON Response
{
  "id": "a1b2c3",
  "name": "Everyday Checking",
  "masked_number": ".... 4471",
  "balance": 4281.16
}

// Swift Model
struct Account: Codable {
    let id: String
    let name: String
    let maskedNumber: String
    let balance: Decimal
}
```



keyDecodingStrategy

- JSONDecoders have a pluggable decoding strategy
- The convertFromSnakeCase strategy handles snake_case -> camelCase automatically for the WHOLE payload

Code Bite

```
let decoder = JSONDecoder()
decoder.keyDecodingStrategy =
    .convertFromSnakeCase

let accounts = try
    decoder.decode([Account].self, from: data)
```



Custom CodingKeys

- Manual mapping - only needed when a key can't be auto-converted
- Simply add the nested enum and the encoders/decoders will take the mappings into account
 - It must be called CodingKeys
 - It must conform to CodingKey and have a String base type

Code Bite

```
struct Account: Codable {  
    let id: String  
    let name: String  
    let maskedNumber: String  
  
    enum CodingKeys: String, CodingKey {  
        case id, name  
        case maskedNumber = "acct_num_masked"  
        // irregular key name  
    }  
}
```



Nested & Optional Fields

```
struct Account: Codable {  
    let id: String  
    let name: String  
    let balance: Decimal  
    let institution: Institution    // nested object  
    let nickname: String?          // optional - may be absent or nil  
}
```

```
struct Institution: Codable {  
    let name: String  
    let routingNumber: String  
}
```

```
// Decoding fails LOUDLY (throws) if a required field is missing -  
// that's a feature, not a bug: better a caught error than silent wrong data.
```



Lesson 3 - JSON Parsing with Codable

Skill Check ✓

- What does `.keyDecodingStrategy = .convertFromSnakeCase` eliminate the need for, in most cases?
- When would you still need a manual `CodingKeys` enum even with `convertFromSnakeCase` enabled?
- Why is it a feature, not a bug, that decoding throws when a required field is missing?



Error Handling & the Generic Fetch Pattern

- Custom NetworkError Types
- A Reusable Generic Fetch Function



Custom NetworkError Types

- Our network code needs to be more specific about error conditions
- Preview: .unauthorized deserves its own case rather than folding into badResponse
 - This lets you trigger an automatic token refresh - a generic error can't carry that meaning

Code Bite

```
enum NetworkError: Error {  
    case invalidURL  
    case noConnection  
    case badResponse(statusCode: Int)  
    case decodingFailed(underlying: Error)  
    case encodingFailed(underlying: Error)  
    case unauthorized  
}
```



Writing Reusable Code

- Repository code we've seen so far has a lot of boilerplate code in each method
- New repository classes for other entities would duplicate that boilerplate
- It is straightforward to encapsulate that boilerplate code in generic methods
- Create a generic base class for all remote repositories and include the boilerplate code
- We can take advantage of this re-write to add more thorough error handling



Generic Repository Base Class

```
class RemoteRepositoryBase<Item: Codable> {  
    func fetchAll(_ urlString: String) async throws -> [Item] {  
        . . .  
    }  
  
    func fetchOne(_ urlString: String) async throws -> Item {  
        . . .  
    }  
  
    func post(_ urlString: String, send item: Item) async throws -> Item { . . . }  
    func put(_ urlString: String, send item: Item) async throws { . . . }  
    func del(_ urlString: String) async throws { . . . }  
}
```



fetchAll() method

```
func fetchAll(_ urlString: String) async throws -> [Item] {  
  
    guard let url = URL(string: urlString) else {  
        throw NetworkError.invalidURL  
    }  
  
    var request = URLRequest(url: url)  
    request.setValue("Bearer \(TokenStore.current)", forHTTPHeaderField: "Authorization")  
  
    let (data, response) = try await URLSession.shared.data(for: request)  
  
    guard let http = response as? HTTPURLResponse else {  
        throw NetworkError.badResponse(statusCode: -1)  
    }  
}
```



fetchAll() method - continued

```
guard (200...299).contains(http.statusCode) else {  
    if http.statusCode == 401 { throw NetworkError.unauthorized }  
    throw NetworkError.badResponse(statusCode: http.statusCode)  
}
```

```
do {  
    let decoder = JSONDecoder()  
    decoder.keyDecodingStrategy = .convertFromSnakeCase  
    return try decoder.decode(Array<Item>.self, from: data)  
} catch {  
    throw NetworkError.decodingFailed(underlying: error)  
}
```

```
}
```



Using the Generic Boilerplate Methods

- Every repository implementation should now inherit from the generic base class
- The inherited boilerplate methods are available to assist in implementing the protocol methods

Code Bite

```
class RemoteProductRepository:
    RemoteRepositoryBase<Product>,
    RepositoryProtocol<Product> {

    func getAll() async throws -> [Product] {
        let urlString = "\(urlBase)/products"
        return try await fetchAll(urlString)
    }

    . . .

}
```



Lesson 4 - Error Handling & the Generic Fetch Pattern

Skill Check ✓

- Why define distinct `NetworkError` cases instead of throwing a single generic error?
- In `fetch<T: Decodable>`, what does the generic parameter `T` let one function do that a non-generic version couldn't?



Auth, Security & Persistence

- Lesson 5 - Authentication Workflows
- Lesson 6 - Secure Token Handling & Keychain
- Lesson 7 - Local Persistence Overview
- Lesson 8 - Core Data Fundamentals I

Authentication Workflows

- Login Flow
- Access & Refresh Tokens
- Automatic Token Refresh on 401



The Login Flow

- 1 App POSTs username/password to /auth/login
- 2 Server validates and returns an access token + a refresh token
- 3 Access token is attached as a Bearer header on every subsequent request
- 4 Access token expires after a short window (commonly 15–60 minutes)
- 5 App uses the refresh token to silently obtain a new access token - no re-login required



Automatic Token Refresh on 401

- If the retry ALSO throws .unauthorized, the refresh token itself has expired
 - That's the one case where the user truly must log in again.

Code Bite

```
func fetchWithAutoRefresh<T: Decodable>(
    _ path: String) async throws -> T {
    do {
        return try await fetch(path)
    } catch NetworkError.unauthorized {
        try await AuthService.shared
            .refreshAccessToken()
        return try await fetch(path)
        // retry once with the new token
    }
}
```



Lesson 5 - Authentication Workflows

Skill Check ✓

- What's the difference in lifespan and purpose between an access token and a refresh token?
- When a request returns 401, what should the app try before assuming the user needs to log in again?
- What signals that even the refresh token has expired and a real re-login is required?



Secure Token Handling & Keychain

- Why UserDefaults Is Not Secure
- Keychain Fundamentals
- A Minimal TokenStore



Why UserDefaults Is Not Secure

- UserDefaults is backed by a plist file stored unencrypted in the app's container.
- It is appropriate for non-sensitive preferences like a display setting.
- It is never appropriate for anything security-sensitive: tokens, passwords, or account numbers.
- @AppStorage is similarly unsecure

Code Bite

```
// DON'T - readable in plain text on disk,  
no encryption  
UserDefaults.standard  
    .set(accessToken, forKey: "authToken")
```



Keychain: What It Actually Provides



Encrypted at rest

Backed by the device's Secure Enclave where available - not just obscured, actually encrypted



Access control options

Can require the device to be unlocked, or even Face ID/Touch ID, to read a stored item



Survives app deletion (by default)

Unlike UserDefaults, Keychain items can persist across a reinstall - a deliberate design choice worth knowing about



KeyChain Access

- The code to interact with secure KeyChain storage is complex
 - It was written in C
- It is best to encapsulate that complexity in a utility class
- The singleton pattern would be a good idea to make access easy
- The KeyChain associates your entries with a string value it calls "service"
 - You should have a unique value for each of your apps

Code Bite

```
import Security

class SecureTokenManager {
    static let shared = SecureTokenManager()

    private init() {}

    private let service = "com.yourapp.auth"
    // this value MUST be unique to your app

}
```



Constructing Queries

- The KeyChain is accessed by running queries
- Queries are constructed as dictionaries
 - The keys are constants defined in the Security module
- The KeyChain stores key/value pairs collected into a "service"
- The keys are referred to as "accounts"

Code Bite

```
let query: [String: Any] = [  
    kSecClass as String:  
        kSecClassGenericPassword,  
    kSecAttrService as String: service,  
    kSecAttrAccount as String: someKey,  
    kSecReturnData as String: true,  
    kSecMatchLimit as String:  
        kSecMatchLimitOne  
]
```



Retrieving Values

- Values are queried from the KeyChain using the SecItemCopyMatching method
- It uses your query dictionary cast as a CFDictionary, and an object reference passed as an inout parameter

Code Bite

```
var dataTypeRef: AnyObject?  
let status = SecItemCopyMatching(  
    query as CFDictionary, &dataTypeRef)  
  
if status == errSecSuccess,  
    let data = dataTypeRef as? Data {  
        return String(data: data,  
            encoding: .utf8)  
    }  
}
```



Deleting and Storing Values

- Deleting a value is straightforward
- So is storing a value
- It is strongly recommended to delete any existing value using a given key (account) before storing a new value for that key
 - This avoids duplicate conflicts

Code Bite

```
// delete a value
SecItemDelete(query as CFDictionary)

// store a value
let status = SecItemAdd(
    query as CFDictionary, nil)
return status == errSecSuccess
```



Lesson 6 - Secure Token Handling & Keychain

Skill Check ✓

- What specifically is wrong with storing an auth token in UserDefaults?
- Name two protections Keychain provides that UserDefaults does not.
- Why wrap the raw Keychain API in a small TokenStore type instead of calling SecItemAdd directly throughout the app?



Local Persistence Overview

- UserDefaults vs. Files vs. Core Data
- Choosing the Right Tool
- Setting Up for Core Data



Choosing the Right Local Storage Tool

Tool	Best For	PNC Mobile Example
UserDefaults	Small, simple, non-sensitive settings	Dark mode preference, last-selected tab
Keychain	Small, sensitive secrets	Auth tokens, biometric-gated secrets
Files (FileManager)	Unstructured blobs	Downloaded PDFs, cached images
Core Data	Structured, relational, queryable data	Accounts, transactions, transfer history



User Defaults

- UserDefaults is a persistent key-value store provided by the Foundation module
- It is designed for saving small amounts of data
 - User preferences, settings, and flags
 - They remain saved even after the app closes and restarts
- It supports String, Int, Double, Float, Bool, Data, Date, URL
 - As well as arrays and dictionaries of those types

Code Bite

```
let defaults = UserDefaults.standard

defaults.set("John Doe", forKey: "username")
defaults.set(25, forKey: "userAge")
defaults.set(true,
              forKey: "hasCompletedOnboarding")
```



Reading and Removing User Defaults

- Reading strings will return an optional
 - Will be nil if key is missing
- Reading non-optional types will return a default value if key is missing
 - e.g. Int, Bool

Code Bite

```
let defaults = UserDefaults.standard
if let username = defaults.string(
    forKey: "username") {
    print("Welcome back, \(username)")
}
let age = defaults.integer(
    forKey: "userAge")
let showTutorial = defaults.bool(
    forKey: "hasCompletedOnboarding")

UserDefaults.standard
    .removeObject(forKey: "username")
```



FileManager

- The FileManager class provides a shared instance to perform file system operations
- Apps running on sandboxed operating systems must store data within predefined sandbox directories
 - You can retrieve these using safe URL properties

Code Bite

```
import Foundation

let fileManager = FileManager.default

let documentsURL = fileManager.urls(
    for: .documentDirectory,
    in: .userDomainMask).first!

let cacheURL =
    fileManager.temporaryDirectory
// Or use .cachesDirectory
```



Creating Files

- To save a file, append a custom filename component to your directory URL and write raw Data to it

Code Bite

```
let fileURL = documentsURL
    .appendingPathComponent("example.txt")
let text = "Hello, Swift File System!"
if let data = text.data(using: .utf8) {
    do {
        try data.write(to: fileURL,
                       options: .atomic)
        print("File saved successfully.")
    }
    catch {
        print("Error writing file: \(error)")
    }
}
```



Reading a File

- Use FileManager to locate or verify the file
- Use Data or String initializers to load the content into memory
- If you are reading plain text, JSON strings, or CSV files, the best approach is to use the String(contentsOf:encoding:) initializer with a file URL

Code Bite

```
let fileURL = documentsURL
    .appendingPathComponent("sample.txt")
if mgr.fileExists(atPath: fileURL.path) {
    do {
        let fileContents = try
            String(contentsOf: fileURL,
                encoding: .utf8)
        print("File Content:\n\(fileContents)")
    } catch {
        print("Error reading file:
            \(error.localizedDescription)")
    }
}
```



Lesson 7 - Local Persistence Overview

Skill Check ✓

- Where should the auth token live: UserDefaults, Keychain, or Core Data? Why?
- Where should transaction history live? Why not just refetch it from the network every time?



Core Data Fundamentals I

- The Core Data Stack
- Building a Data Model Together
- NSManagedObjectContext Basics



The Core Data Stack



NSPersistentContainer

Sets up and owns the whole Core Data stack - the entry point



NSManagedObjectContext

Your in-memory 'scratchpad' - all reads, inserts, and edits happen here



Persistent Store

The actual file on disk - SQLite by default, invisible day-to-day

Code Bite

```
let container = NSPersistentContainer(  
    name: "PNCMobileApp")  
  
container.loadPersistentStores { _, error in  
    if let error {  
        fatalError(  
            "Core Data failed to load:\(error)"  
        )  
    }  
}  
  
let context = container.viewContext
```



Building the Data Model Together (Live)

- 1 Add a new .xcdatamodeld file to the project (or open the existing one)
- 2 Add Entity → name it AccountEntity
- 3 Add attributes: id (String), name (String), maskedNumber (String), balance (Decimal)
- 4 Set Codegen to 'Class Definition' so Xcode generates the NSObject subclass automatically
- 5 Build the project - confirm AccountEntity is now a usable Swift type



Lesson 8 - Core Data Fundamentals I

Skill Check ✓

- What's the difference in role between `NSPersistentContainer` and `NSManagedObjectContext`?
- What file format actually backs the persistent store by default?
- What does setting Codegen to 'Class Definition' do for you?



Core Data & Caching

- Lesson 9 - Core Data Fundamentals II: CRUD & Relationships
- Lesson 10 - Caching Strategies
- Module Exercise - The PNC Mobile Accounts Data Layer

Core Data Fundamentals II

- Create, Fetch, Update, Delete
- NSFetchRequest & Predicates
- Relationships



Create New Items in CoreData

- CoreData entity objects are creatable just like any other Swift objects
- They need the data context passed into their initializer, however
 - This allows the context to track their state
- They only exist in memory until the context's save method is called

Code Bite

```
// Create
let entity = AccountEntity(context: context)
entity.id = "a1b2c3"
entity.name = "Everyday Checking"
entity.balance = 4281.16

// Nothing is written to disk until:
try context.save()
```



Fetch Items Using CoreData

- Each entity type has a `fetchRequest` method that creates a Generic `NSFetchRequest` object
- Requests can use an `NSPredicate` to filter the results
- `NSSortDescriptors` can be used to sort the results
 - The `sortDescriptors` property is an array to provide for multi-level sort
- The `fetchLimit` property can throttle the number of results returned

Code Bite

```
let request = Employee.fetchRequest()

request.predicate = NSPredicate(format:
    "department == %@", "Engineering")

request.sortDescriptors = [NSSortDescriptor(
    key: "lastName", ascending: true)]

request.fetchLimit = 50

let employees = try context.fetch(request)
```



Update Items Using CoreData

- Objects loaded by a CoreData context are tracked by that context
- Changes to their properties are flagged by the context
- When the context's save method is called any tracked object that has had a property changes will be updated in the database

Code Bite

```
// Update - mutate a fetched object directly  
employees.first?.salary = 125_000.00
```

```
// Nothing is written to disk until:  
try context.save()
```



Delete Items Using CoreData

- You can pass any tracked object to the context's delete method
- The deletion is not persisted to the database until the context's save method is called

Code Bite

```
let products = try context.fetch(requests)

for product in products {
    context.delete(product)
}

// Nothing is written to disk until:
try context.save()
```



NSFetchRequest & Predicates

- Predicates are evaluated by the persistent store itself, not by fetching everything into memory and filtering in Swift
 - This matters a great deal once a table has thousands of rows.

Code Bite

```
let request = AccountEntity.fetchRequest()
request.predicate = NSPredicate(
    format: "balance < %@",
    NSNumber(decimal: 0))

request.sortDescriptors =
    [NSSortDescriptor(key: "balance",
                      ascending: true)]

let overdraftAccounts = try
    context.fetch(request)

// Equivalent SQL query: SELECT * FROM
// Account WHERE balance < 0 ORDER BY balance
// ASC
```



Relationships

```
// In the model editor: Account <--> Transaction  
// (one account has many transactions; each transaction belongs to one account)
```

```
let transaction = Transaction(context: context)  
transaction.amount = -42.50  
transaction.date = Date()  
transaction.account = account    // sets both sides of the relationship  
  
// Traverse from the other direction:  
let allTransactionsForAccount = account.transactions
```



Lesson 9 - Core Data Fundamentals II

Skill Check ✓

- At what exact point does a Core Data change actually get written to disk?
- Why is filtering with NSPredicate preferable to fetching everything and filtering in Swift?
- Once a relationship is set on one side (transaction.account = accountEntity), what happens on the other side?



Caching Data

- Caching Strategies
- Cache Invalidation
- ETags & Conditional Requests



Caching Strategies

- Cache First
 - Check local cache; if not there, load from network
 - Good for static assets such as user profiles
- Network First
 - Load from network; if error, load from local cache
 - Useful for offline support, if stale data is more acceptable than app unavailability
- Cache-Then-Network
 - Load from local cache, then load from network, store results in cache and update onscreen
 - Provides fast initial view of (possibly stale) data, with eventual refresh of data



Cache-Then-Network, Concretely

- 1 Repository immediately returns whatever is currently in the cache (may be stale or empty)
- 2 UI renders that cached data right away - no spinner-first experience
- 3 Repository kicks off a network fetch in the background
- 4 When the network response arrives, it's saved to the cache
- 5 The UI observes the cache change and updates again, now with fresh data



Cache in Memory

- Caching in memory is the fastest to load
 - Also the most volatile
- A dictionary could work
- `NSCache<Tkey, Tvalue>` is a better choice
 - It will automatically evict items when low on memory
- It only works with object references
 - Hence `NSString` for the keys

Code Bite

```
let cache = NSCache<NSString, Customer>()

// store objects
cache.setObject(cust,
                forKey: "ABC123" as NSString)

// retrieve objects
let myCust = cache.object(
    forKey: "ABC123" as NSString)
```



Cache on Disk

- Disk storage can be a valid cache destination
 - Data will survive application shutdown and device restarting
- FileManager can be used to store data directly to files
 - Raw Data or Codable objects can be written directly to local storage
- Core Data can be used to store objects to a database
- Some third-party modules are specifically designed to provide caching
 - HyperOslo Cache and Mobelux DiskCache are popular choices
- Caching strategies can involve caching in memory, caching on disk, and network calls



Cache Invalidation & ETags

```
// First request - server returns data plus an ETag fingerprint
```

```
// Response header: ETag: "a1b2c3-v7"
```

```
// Subsequent request - send the fingerprint back
```

```
request.setValue(storedETag, forHTTPHeaderField: "If-None-Match")
```

```
let (data, response) = try await URLSession.shared.data(for: request)
```

```
if let http = response as? HTTPURLResponse, http.statusCode == 304 {
```

```
    // 304 Not Modified - nothing changed, use the existing cached data,
```

```
    // and we didn't even pay the cost of re-downloading the full payload
```

```
}
```



Cache First Example

```
func fetchAccounts() async throws -> [Account] {  
    let cached = try fetchFromCoreData()  
  
    Task { [weak self] in  
        guard let self else { return }  
        do {  
            let dtos: [AccountDTO] = try await fetch("/accounts")  
            try self.saveToCoreData(dtos)  
        } catch {  
            // nothing - do NOT want to throw in here  
        }  
    }  
  
    return cached  
}
```



Lesson 10 - Caching Strategies

Skill Check ✓

- In cache-then-network, why does the UI update twice for a single screen load?
- What does a 304 Not Modified response mean, and why is it valuable beyond just saving a re-parse?
- How does this pattern connect back to Module 7's offline-first design concepts?



Module 8 Complete

Across three days you built a complete, real data layer: REST networking with async/await, typed JSON parsing with Codable, secure token storage in Keychain, automatic token refresh, a full Core Data stack with relationships, and a cache-then-network Repository that makes the app usable offline. Every piece from Module 7's architecture is now filled in with a real implementation.

Next: Module 9 - Testing & Debugging for iOS